

ОДЕСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ І. І. МЕЧНИКОВА
ФАКУЛЬТЕТ МАТЕМАТИКИ, ФІЗИКИ І ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ
КАФЕДРА ОПТИМАЛЬНОГО КЕРУВАННЯ І ЕКОНОМІЧНОЇ КІБЕРНЕТИКИ

Паралельні алгоритми обчислювальної математики

Звіт з лабораторної роботи №3 Топологія процесів MPI

Виконав: студент 1 курсу магістратури
спеціальності 113 Прикладна математика
Чачко Натан Леонідович

2025 р.

Теорія

Топологія процесів — це структура, що визначає, як процеси у програмі зв'язуються один з одним і як між ними здійснюється передача даних. Вона може бути різною в залежності від задачі та архітектури комп'ютерної системи. Наприклад, топологія може бути лінійною, багатовимірною, з кількома вимірами, або ієрархічною.

Однією з основних переваг використання топологій процесів є те, що вони дозволяють оптимізувати маршрути передачі даних, зменшуючи затримки і навантаження на систему. У системах, що підтримують MPI (Message Passing Interface), для створення топологій існують спеціальні функції, такі як `MPI_Cart_create`, які дозволяють організовувати процеси у вигляді багатовимірних решіток.

Основні функції для роботи з топологіями процесів:

- `MPI_Init` — функція, яка ініціалізує середовище MPI, необхідне для подальшого використання MPI-функцій.
- `MPI_Cart_create` — функція, що створює картезіанську топологію для комунікації між процесами.
- `MPI_Comm_rank` — функція, яка визначає ранг процесу в заданому комунікаторі.
- `MPI_Cart_coords` — функція, яка дозволяє отримати координати процесу в картезіанській топології.
- `MPI_Cart_rank` — функція, що визначає ранг процесу в картезіанській топології за його координатами.
- `MPI_Recv` — функція, яка приймає повідомлення від іншого процесу.
- `MPI_Send` — функція, яка відправляє повідомлення іншому процесу.
- `MPI_Finalize` — функція, що завершує роботу з MPI і звільняє ресурси.

Завдяки цим функціям можна легко створювати і маніпулювати різними типами топологій, що значно спрощує розробку та оптимізацію паралельних програм.

Картезіанська топологія в контексті паралельних обчислень з використанням MPI — це тип топології процесів, де процеси розташовані в багатовимірному просторі, і кожен процес має координати, які визначають його позицію в цьому просторі.

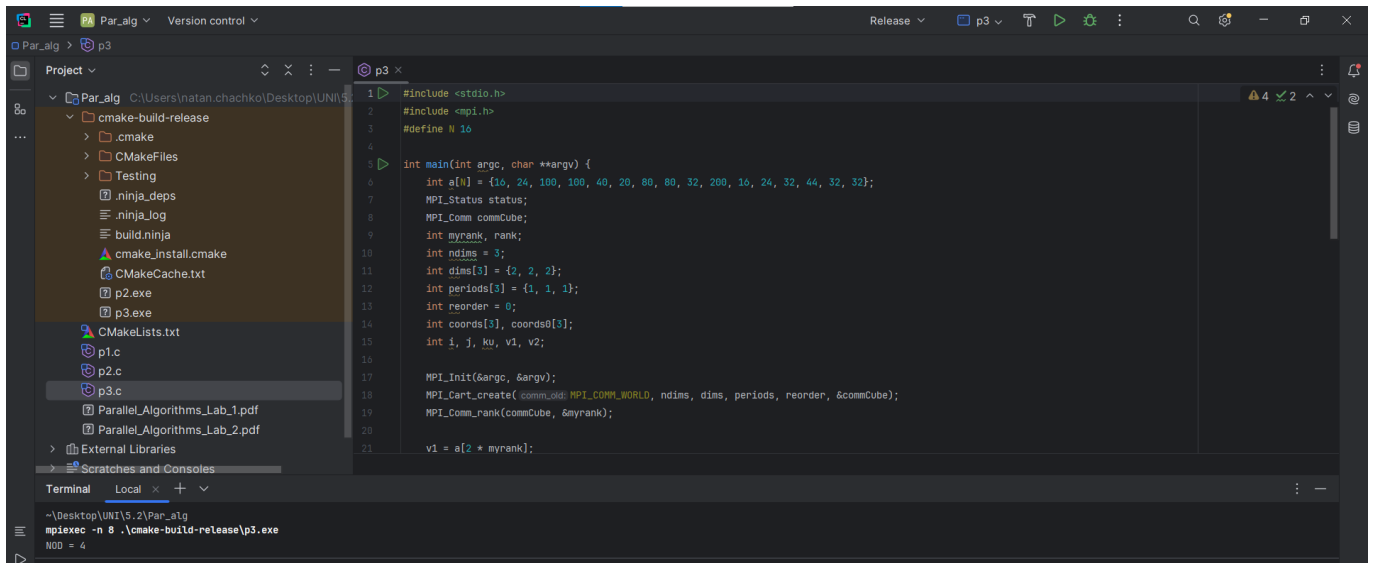
Завдання

Написати MPI-додаток (використовуючи декартову топологію процесів) на вирішення наступного завдання. Нехай $N = 2^n$ процесів розміщені у вершинах одиничного гіперкубу простору R^n ($n > 2$). Кожен процес може взаємодіяти тільки з n процесами за жорстко закріпленими лініями зв'язку з номерами від 1 до n (лінії зв'язку є ребрами гіперкубу). Набір координат $u = (u_1, \dots, u_n)$ (де $u_i \in \{0, 1\}$ для всіх $1 \leq i \leq n$) довільної вершини гіперкуба задає ім'я процесу, що відповідає цій вершині. $N = 2^n$ паралельних процесів з топологією n -вимірного гіперкуба знаходять найбільший спільний дільник 2^{n+1} цілих чисел.

До звіту включити текст програми та скрін-шоти компіляції та виконання MPI-додатка.

```
1 #include <stdio.h>
2 #include <mpi.h>
3 #define N 16
4
5 int main(int argc, char **argv) {
6     int a[N] = {16, 24, 100, 100, 40, 20, 80, 80, 32, 200, 16, 24, 32, 44, 32, 32};
7     MPI_Status status;
8     MPI_Comm commCube;
9     int myrank, rank;
10    int ndims = 3;
11    int dims[3] = {2, 2, 2};
12    int periods[3] = {1, 1, 1};
13    int reorder = 0;
14    int coords[3], coords0[3];
15    int i, j, ku, v1, v2;
16
17    MPI_Init(&argc, &argv);
18    MPI_Cart_create(MPI_COMM_WORLD, ndims, dims, periods, reorder, &commCube);
19    MPI_Comm_rank(commCube, &myrank);
20
21    v1 = a[2 * myrank];
22    v2 = a[2 * myrank + 1];
23    MPI_Cart_coords(commCube, myrank, ndims, coords);
24    i = 0;
25    while ((i < ndims) && (coords[i] == 0)) i++;
26    ku = i + 1;
27    for (i = 1; i < ku; i++) {
28        while (v1 != v2)
29            if (v1 > v2) v1 -= v2;
30            else v2 -= v1;
31        for (j = 0; j < ndims; j++) coords0[j] = coords[j];
32        coords0[i - 1] = (coords0[i - 1] + 1) % 2;
33        MPI_Cart_rank(commCube, coords0, &rank);
34        MPI_Recv(&v2, 1, MPI_INT, rank, 99, commCube, &status);
35    }
36    while (v1 != v2)
37        if (v1 > v2) v1 -= v2;
38        else v2 -= v1;
39    if (ku <= ndims) {
40        for (j = 0; j < ndims; j++) coords0[j] = coords[j];
41        coords0[i - 1] = (coords0[i - 1] + 1) % 2;
42        MPI_Cart_rank(commCube, coords0, &rank);
43        MPI_Send(&v1, 1, MPI_INT, rank, 99, commCube);
44    } else
45        printf("NOD = %d\n", v1);
46
47    MPI_Finalize();
48 }
```

Результат роботи програми:



```
1 #include <stdio.h>
2 #include <mpi.h>
3 #define N 16
4
5 int main(int argc, char **argv) {
6     int a[N] = {16, 24, 100, 100, 40, 20, 80, 80, 32, 200, 16, 24, 32, 44, 32, 32};
7     MPI_Status status;
8     MPI_Comm commCube;
9     int myrank, rank;
10    int ndims = 3;
11    int dims[3] = {2, 2, 2};
12    int periods[3] = {1, 1, 1};
13    int reorder = 0;
14    int coords[3], coords0[3];
15    int i, j, ku, v1, v2;
16
17    MPI_Init(&argc, &argv);
18    MPI_Cart_create(MPI_COMM_WORLD, ndims, dims, periods, reorder, &commCube);
19    MPI_Comm_rank(commCube, &myrank);
20
21    v1 = a[2 * myrank];
```

Terminal Local x + v

```
~\Desktop\UNI\5.2\Par_alg
mpirun -n 8 .\cmake-build-release\p3.exe
W00 = 4
```

Обчислення проводилися з використанням компілятора GCC (MinGW) на комп'ютері з операційною системою Windows 10 та наступними характеристиками:

CPU:

11th Gen Intel(R) Core(TM) i7-1185G7 @ 3.00GHz 1.80 GHz

Base speed: 1,80 GHz

Cores: 4

Logical processors: 8

Для запуску програми необхідно було встановити [Microsoft MPI \(MS-MPI\)](#). MS-MPI є реалізацією стандарту інтерфейсу передачі повідомлень (MPI) від Microsoft, яка дозволяє розробляти та виконувати паралельні програми на платформі Windows.

Після встановлення MPI необхідно налаштувати середовище для запуску програми. У випадку з Microsoft MPI це включає:

- Додавання шляху до директорії, де розташовані виконувані файли MPI, до системної змінної PATH.
- Використання команд `mpicc` для компіляції програм з підтримкою MPI та `mpirun` для їх запуску.

Після налаштування середовища можна було скомпілювати програму за допомогою компілятора GCC (MinGW) та виконати паралельні обчислення на декількох процесорах.

У результаті тестування програма працювала коректно, виконуючи обчислення з використанням кількох процесів, що дозволяє значно прискорити обробку великих обсягів даних.